GPT-Style Transformer Decoder: Complete Numerical Calculation

We will follow the entire calculation from input words → tokens → embeddings → Q/K/V → attention scores → causal masking → 10×10 heatmap → contextual vectors → logits → next-token probabilities.

Our example is:

> The apple is read, it is really good for health

I have retained "read" exactly as typed. If you intended "red," we can substitute it later. For this calculation, punctuation is separated and omitted, giving exactly 10 word tokens.

1. Tokenization and vocabulary

Position	Word	Illustrative token ID
1	The	101
2	apple	205
3	is	53
4	read	314
5	it	88
6	is	53
7	really	276
8	good	246
9	for	67
10	health	412

A word's token ID identifies an entry in the embedding dictionary. These are invented IDs for the exercise; they are not GPT tokenizer IDs. We also intentionally use word-level rather than subword tokenization.

2. Choose the model dimensions

To keep the arithmetic manageable, we will build a miniature single-head causal-attention model.

Parameter	Value	Meaning
N	10	Input tokens
d_{model}	2	Numbers in each token vector
h	1	Attention head
d_k	2	Query/key dimensions
d_v	2	Value-vector dimensions
Vocabulary output size	4	Four candidate next tokens

These tiny dimensions allow us to calculate all 100 heatmap cells. Real GPT models normally have substantially larger dimensions, many heads, multiple Transformer blocks and trained parameters.

3. From embeddings to input vectors

The input to the attention layer is:

$$\boxed{X_i = E_i + P_i}$$

where E_i is the token embedding and P_i is the positional encoding.

For this worked example, we use the illustrative positional rule:

$$P_i = [0.01i,\ 0.02i]$$

This is a deliberately simple teaching rule, not the sinusoidal or learned positional encoding normally used in Transformer architectures.

For apple, at position 2:

$$E_{\text{apple}} = [0.78,\ 0.16]$$

$$P_2 = [0.02,\ 0.04]$$

Therefore,

$$X_{\text{apple}} = [0.78 + 0.02,\ 0.16 + 0.04]$$

$$\boxed{X_{\text{apple}} = [0.80,\ 0.20]}$$

The same procedure is applied to all 10 words.

Token	Embedding E_i	Position P_i	Final X_i
The	[0.19, 0.08]	[0.01, 0.02]	[0.20, 0.10]
apple	[0.78, 0.16]	[0.02, 0.04]	[0.80, 0.20]
is	[0.07, 0.24]	[0.03, 0.06]	[0.10, 0.30]
read	[0.66, 0.12]	[0.04, 0.08]	[0.70, 0.20]
it	[0.25, 0.10]	[0.05, 0.10]	[0.30, 0.20]
is	[0.04, 0.18]	[0.06, 0.12]	[0.10, 0.30]
really	[0.13, 0.36]	[0.07, 0.14]	[0.20, 0.50]
good	[0.32, 0.44]	[0.08, 0.16]	[0.40, 0.60]
for	[0.11, 0.02]	[0.09, 0.18]	[0.20, 0.20]
health	[0.50, 0.50]	[0.10, 0.20]	[0.60, 0.70]

Thus our input matrix is:

$$X = \begin{bmatrix} 0.2 & 0.1 \\ 0.8 & 0.2 \\ 0.1 & 0.3 \\ 0.7 & 0.2 \\ 0.3 & 0.2 \\ 0.1 & 0.3 \\ 0.2 & 0.5 \\ 0.4 & 0.6 \\ 0.2 & 0.2 \\ 0.6 & 0.7 \end{bmatrix} \quad (10 \times 2)$$

4. Calculate Query, Key and Value vectors

The neural-network projections are:

$$Q = XW_Q + b_Q$$

$$K = XW_K + b_K$$

$$V = XW_V + b_V$$

To simplify the first experiment, set:

$$W_Q = W_K = W_V = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

and all biases to zero.

Therefore:

$$\boxed{Q = K = V = X}$$

For apple:

$$Q_{\text{apple}} = [0.8,0.2]$$

$$K_{\text{apple}} = [0.8,0.2]$$

$$V_{\text{apple}} = [0.8, 0.2]$$

This equality occurs only because we chose identity projection matrices. Normally, Q, K and V are different learned projections.

5. Calculate raw attention scores

The attention score between query position i and key position j is:

$$\boxed{S_{ij} = \frac{Q_i \cdot K_j}{\sqrt{d_k}}}$$

Here:

$$d_k = 2, \quad \sqrt{d_k} = 1.414214$$

Let us take apple as the query.

Against the first token, The:

$$S_{2,1} = \frac{(0.8)(0.2) + (0.2)(0.1)}{1.414214}$$

$$= \frac{0.18}{1.414214} = \boxed{0.1273}$$

Against its own key, apple:

$$S_{2,2} = \frac{(0.8)(0.8) + (0.2)(0.2)}{1.414214}$$

$$= \frac{0.68}{1.414214} = \boxed{0.4808}$$

Against read, at position 4:

$$S_{2,4} = \frac{(0.8)(0.7) + (0.2)(0.2)}{1.414214}$$

$$= \frac{0.60}{1.414214} = \boxed{0.4243}$$

The numerical score for read is available at this stage, but the decoder must mask it because it comes after apple.

6. Apply the causal mask

The GPT-style decoder cannot use future words when processing a particular position.

$$M_{ij} = \begin{cases} 0, & j \le i \\ -\infty, & j > i \end{cases}$$

For query apple, position 2, only positions 1 and 2 remain accessible.

$$S'_{\text{apple}} = [0.1273,\ 0.4808,\ -\infty, \ldots, -\infty]$$

7. Calculate softmax for apple

$$A_{ij} = \frac{e^{S'_{ij}}}{\sum_k e^{S'_{ik}}}$$

For the two allowed positions:

$$A_{2,1} = \frac{e^{0.1273}}{e^{0.1273} + e^{0.4808}} = \boxed{0.4125}$$

$$A_{2,2} = \frac{e^{0.4808}}{e^{0.1273} + e^{0.4808}} = \boxed{0.5875}$$

All future positions receive zero attention.

$$\boxed{A_{\text{apple}} = [0.4125, 0.5875, 0, 0, 0, 0, 0, 0, 0, 0]}$$

This is the second row of our 10×10 decoder attention heatmap.

8. Complete 10×10 attention heatmap

Repeating the Q–K dot product, causal mask and softmax for all ten query positions gives the following matrix.

Values rounded to four decimal places; column headings are key-token positions. Each row is a query token.

GPT-style causal attention — 10 queries × 10 keys

Q ↓ / K →	The 1	apple 2	is 3	read 4	it 5	is 6	really 7	good 8	for 9	health 10
The 1	1.000	0	0	0	0	0	0	0	0	0
apple 2	0.413	0.587	0	0	0	0	0	0	0	0
is 3	0.322	0.344	0.334	0	0	0	0	0	0	0
read 4	0.215	0.294	0.211	0.280	0	0	0	0	0	0
it 5	0.188	0.216	0.189	0.212	0.195	0	0	0	0	0
is 6	0.161	0.171	0.166	0.170	0.165	0.166	0	0	0	0
really 7	0.132	0.149	0.140	0.147	0.139	0.140	0.152	0	0	0
good 8	0.110	0.136	0.116	0.132	0.118	0.116	0.130	0.143	0	0
for 9	0.106	0.117	0.108	0.115	0.109	0.108	0.112	0.117	0.108	0
health 10	0.082	0.112	0.087	0.107	0.090	0.087	0.101	0.115	0.087	0.132

Higher attention · Lower attention · Causally masked

Selected query: apple (position 2)

Row sum = 1.000000

Select any row in the matrix to examine its calculated attention distribution.

The: 0.4125; apple: 0.5875; is: 0.0000; read: 0.0000; it: 0.0000; is: 0.0000; really: 0.0000; good: 0.0000; for: 0.0000; health: 0.0000.

Heatmap values are calculated from the exact input matrix and identity Q/K weights stated above, not manually assigned. The lower-triangular structure comes from causal masking. These untrained values do not demonstrate linguistic understanding.

Why is the heatmap triangular?

- Row 1 (The): Can attend only to position 1; attention = 1.
- Row 2 (apple): Can attend to positions 1–2.
- Row 5 (it): Can attend to positions 1–5.
- Row 10 (health): Can attend to all positions 1–10.

In contrast, a bidirectional encoder would generally calculate attention over all ten valid key positions for each query, producing a full 10×10 matrix without the causal zero region.

9. Calculate the attention-weighted Value vectors

The attention heatmap alone is not the Transformer output. The next operation is:

$$\boxed{Z = AV}$$

For token apple:

$$Z_{\text{apple}} = 0.4125 V_{\text{The}} + 0.5875 V_{\text{apple}}$$

Recall:

$$V_{\text{The}} = [0.2, 0.1], \quad V_{\text{apple}} = [0.8, 0.2]$$

Therefore,

$$Z_{\text{apple}} = 0.4125[0.2,0.1] + 0.5875[0.8,0.2]$$

$$= [0.0825 + 0.4700,\ 0.04125 + 0.1175]$$

$$\boxed{Z_{\text{apple}} \approx [0.5525,\ 0.1587]}$$

The same weighted combination is calculated for each query.

Token	Context vector Z_i
The	[0.2000, 0.1000]
apple	[0.5525, 0.1587]
is (3)	[0.3728, 0.2012]
read	[0.4952, 0.1995]
it	[0.4363, 0.2001]
is (6)	[0.3710, 0.2172]
really	[0.3490, 0.2605]
good	[0.3645, 0.3086]
for	[0.3408, 0.2914]
health	[0.3878, 0.3511]

These vectors are obtained by multiplying the 10×10 attention matrix by the 10×2 Value matrix:

$$\underset{10\times10}{A}\ \underset{10\times2}{V} = \underset{10\times2}{Z}$$

The contextual vector is what carries the combined information from accessible token positions.

For our deliberately simplified example, we will use Z directly as input to a vocabulary projection. A complete GPT Transformer block would also contain an attention output projection, residual connections, normalization and a feed-forward network, often repeated across many blocks. We are omitting those components from this numerical exercise, so the calculations are reproducible without additional parameter matrices.

10. From final contextual vector to vocabulary logits

Now we arrive at the central step of GPT-style text generation.

After processing the complete 10-token prefix, the decoder uses the final token position, health, to calculate the next-token distribution.

Our calculated final attention output is:

$$Z_{\text{health}} = [0.3878,\ 0.3511]$$

Step A — Define the vocabulary projection

For demonstration, suppose the next-token vocabulary has four candidates:

$$\mathcal{V} = \{\text{fruit,healthy,.,today}\}$$

Choose the following illustrative vocabulary weight matrix:

$$W_{\text{vocab}} = \begin{bmatrix} 1 & 0 & -1 & 0 \\ 0 & 1 & 0 & -1 \end{bmatrix}$$

and bias:

$$b_{\text{vocab}} = [0.1,0,0,0]$$

The dimensions are:

$$(1 \times 2)(2 \times 4) + (1 \times 4) = (1 \times 4)$$

Step B — Calculate logits

$$\boxed{L = Z_{\text{health}} W_{\text{vocab}} + b_{\text{vocab}}}$$

Substituting the values:

$$L = [0.3878,\ 0.3511] \begin{bmatrix} 1 & 0 & -1 & 0 \\ 0 & 1 & 0 & -1 \end{bmatrix} + [0.1,0,0,0]$$

$$L = [0.3878,\ 0.3511,\ -0.3878,\ -0.3511] + [0.1,0,0,0]$$

$$\boxed{L = [0.4878,\ 0.3511,\ -0.3878,\ -0.3511]}$$

These four numbers are called logits. They are unnormalized scores, not probabilities.

11. Convert logits into next-token probabilities

Apply softmax:

$$P(w_j) = \frac{e^{L_j}}{\sum_{k=1}^{4} e^{L_k}}$$

For fruit:

$$P(\text{fruit}) = \frac{e^{0.4878}}{e^{0.4878} + e^{0.3511} + e^{-0.3878} + e^{-0.3511}}$$

$$\boxed{P(\text{fruit}) \approx 0.3675}$$

Similarly, calculate the other three probabilities.

Calculated next-token distribution

Token	Probability	Logit
fruit	36.75%	0.4878
healthy	32.06%	0.3511
today	15.88%	-0.3511
.	15.31%	-0.3878
Total probability	**100.00%**	

If we apply greedy decoding—selecting the candidate with the highest probability—the next token will be:

$$\boxed{\text{fruit}}$$

Therefore, the hypothetical model continues:

> The apple is read, it is really good for health fruit

This is grammatically poor. That is intentional: it demonstrates that numerically correct attention and logits do not imply learned linguistic competence.

With actual training, the embedding, Q/K/V, neural-network and vocabulary projection weights would be adjusted through backpropagation to predict tokens more appropriately.

12. Complete calculation chain

Stage	Computation / result
INPUT	10 word tokens
TOKEN LOOKUP	Vocabulary IDs → E (10×2)
POSITION	X = E + P (10×2)
Q / K / V	Three linear projections (10×2 each)
ATTENTION SCORES	$QK^T / \sqrt{2}$ (10×10)
CAUSAL MASK	Block all future positions

Stage	Computation / result
HEATMAP	Row-wise Softmax (10×10)
CONTEXTUAL OUTPUT	Z = AV (10×2)
FINAL-POSITION PROJECTION	z_{10} × W_vocab + b (1×4)
SOFTMAX	Four next-token probabilities
GENERATE	Choose fruit, append, repeat